

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВИЙ ПРОЄКТ
з освітнього компоненту «Програмування»
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «БУФЕТ»

Студента 1 курсу, денна форма навчання
спеціальність F7 «Комп'ютерна інженерія»

Янчевського Кирила Сергійовича

Керівник: к.ф.- м.н., доц. Антоненко О. С.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____ Антоненко О.С.
(підпис) (прізвище та ініціали)

_____ Шестопалов С.В.
(підпис) (прізвище та ініціали)

_____ _____
(підпис) (прізвище та ініціали)

ЗМІСТ

	стор.
ВСТУП	3
ЗАВДАННЯ	4
1. ТЕОРЕТИЧНА ЧАСТИНА	5
1.1 Поняття ООП.....	5
1.2 Опис об'єктної частини.....	6
2. ПРАКТИЧНА ЧАСТИНА	7
2.1 Опис класів	7
2.2 Інтерфейс.....	11
ВИСНОВКИ.....	19
СПИСОК ЛІТЕРАТУРИ.....	20
Додаток А. Діаграма класів.....	21
Додаток Б. Код основних класів.....	22

ВСТУП

Метою цієї роботи є розробка ієрархії класів для моделювання роботи «Буфету» та її застосування для вирішення конкретної прикладної задачі згідно з варіантом. У процесі проектування системи важливо практично реалізувати ключові концепції об'єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм. Кінцевий програмний продукт має представляти собою систему взаємопов'язаних класів, які детально описують внутрішню структуру та бізнес-процеси буфету.

Для коректної роботи з об'єктами необхідно передбачити відповідні конструктори, поля та методи. Доступ до полів класу має бути закритим, а взаємодія з ними повинна відбуватися виключно через публічні методи. Структура програми має базуватися на зв'язках успадкування та композиції. Окрім цього, для забезпечення динамічного поліморфізму слід інтегрувати механізм віртуальних методів, а також задіяти абстрактні класи чи чисті віртуальні функції там, де це необхідно.

ЗАВДАННЯ

Варіант 23б. Створення ієрархії класів на тему «Буфет»

У роботі необхідно реалізувати класи, що описують структуру буфету, меню страв, відвідувачів та продавців. Потрібно передбачити можливість роботи з різними категоріями страв, збереження інформації про їхню назву, ціну, доступну кількість. Також необхідно реалізувати взаємодію між класами, перевірку коректності введених даних та виконання основних операцій, пов'язаних із функціонуванням буфету: процеси замовлення, оплати та контролю.

1. ТЕОРЕТИЧНА ЧАСТИНА

1.1 Поняття ООП

Об'єктно-орієнтований програмування (ООП) – це парадигма розробки, де програма будується як мережа взаємодіючих об'єктів [1]. Будь-який об'єкт є екземпляром певного класу, а самі класи організовані в ієрархічну структуру. Спочатку розробник описує класи, на базі яких під час виконання коду генеруються конкретні об'єкти. Завдяки можливості розширювати базові класи новими, вибудовується розгалужена класова ієрархія [2].

Фундаментом цього стилю програмування є об'єктна модель, яка тримається на трьох головних концепціях [2]:

- а) інкапсуляція;
- б) наслідування;
- с) поліморфізм.

Інкапсуляція: Цей механізм полягає в об'єднанні інформації (даних) та функцій (методів), що з нею працюють, у межах одного об'єкта. Це забезпечує надійний захист внутрішньої структури від небажаного зовнішнього втручання або помилкового використання [1,2].

Наслідування: Цей процес надає можливість одному об'єкту переймати властивості та поведінку іншого. Таким чином, об'єкт може не лише копіювати існуючі характеристики, але й доповнювати їх своїми унікальними рисами [1,2].

Поліморфізм: Це властивість, що дозволяє застосовувати одне й те саме ім'я для опису спільного класу дій, вирішуючи при цьому різні технічні завдання. На практиці це означає, що об'єкт здатний самостійно обирати потрібний внутрішній метод обробки інформації, орієнтуючись на тип даних, які він отримує у повідомленні [1,2].

1.2 Опис об'єктної частини

Програма пропонує систему розподілу прав для трьох категорій користувачів: відвідувач, продавець та інспектор. Можливість виконання тих чи інших дій всередині програми безпосередньо залежить від вибору відвідувача та продавця користувачем, перевірки його балансу (для здійснення замовлення їжі чи напоїв).

Відвідувач взаємодіє з інтерфейсом буфету для перегляду актуального меню (асортименту гарячих страв, холодних страв та напоїв) та здійснення покупки. Він купує продукцію, якщо його баланс є достатнім для оплати обраних позицій, чи, якщо баланс є недостатнім, має вибір скоригувати замовлення або зупинити вибір страв.

Головним елементом системи є меню буфету, що складається з базового абстрактного класу страв та трьох спеціалізованих класів-нащадків: гарячі страви, холодні страви та напої. Кожна позиція меню характеризується набором закритих параметрів (назва, вартість, поточна кількість товару). Робота із асортиментом здійснюється безпосередньо через інтерфейс програми, який дозволяє переглядати доступні товари та перевіряти наявність необхідної кількості одиниць.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Діаграму класів наведено в додатку А. Розглянемо структуру кожного класу.

- 1) Абстрактний клас «Person» – людина:
 - a) Захищені поля: fullName, position.
 - b) Конструктори:
 - Конструктор без параметрів;
 - Конструктор з параметрами.
 - c) Методи:
 - Чисто віртуальна функція printInfo.
- 2) Клас «Visitor», нащадок класу «Person» – відвідувач:
 - a) Приватні поля: money.
 - b) Конструктори:
 - Конструктор без параметрів;
 - Конструктор з параметрами.
 - c) Методи:
 - getMoney – повертає кількість грошей об'єкта;
 - changeMoney – змінює кількість грошей об'єкта;
 - Перевизначення printInfo – виводить інформацію про об'єкт.
- 3) Клас «Seller», нащадок класу «Person» – продавець:
 - a) Приватні поля: badProbability, stolen, cheated, arrested.
 - b) Конструктори:
 - Конструктор без параметрів;
 - Конструктор з параметрами.

с) Методи:

- getProb – повертає шанс шахрайства;
- stole – повертає скільки вкрадено;
- isArrested – повертає чи арестован об'єкт;
- testCheat – повертає чи зшахрував об'єкт;
- cheat – змінює поле cheated на true;
- getArrested – змінює поле arrested на true;
- steal – змінює кількість вкрадених грошей;
- printName – виводить ім'я об'єкта;
- Перевизначення printInfo – виводить інформацію про об'єкт.

4) Клас «Inspector», нащадок класу «Person» – інспектор:

a) Приватні поля: organization.

b) Конструктори:

- Конструктор без параметрів;
- Конструктор з параметрами.

с) Методи:

- Перевизначення printInfo – виводить інформацію про об'єкт.

5) Абстрактний клас «Dish» – страва:

a) Приватні поля: name, price, quantity, tempSold.

b) Конструктори:

- Конструктор без параметрів;
- Конструктор з параметрами.

с) Методи:

- getName – повертає назву об'єкта;
- getPrice – повертає вартість об'єкта;
- getQuantity – повертає кількість порцій об'єкта;
- getTempSold – повертає поле tempSold;

- increaseQuantity – збільшує кількість порцій об'єкта;
 - increaseTempSold – збільшує значення поля tempSold;
 - resetTempSold – змінює значення поля tempSold на 0;
 - printInfo – виводить інформацію про об'єкт;
 - Чисто віртуальна функція getType.
- 6) Клас «HotDish», нащадок класу «Dish» – гаряча страва:
- a) Конструктори:
 - Конструктор з параметрами.
 - b) Методи:
 - Перевизначення getType – повертає вид страви.
- 7) Клас «ColdDish», нащадок класу «Dish» – холодна страва:
- a) Конструктори:
 - Конструктор з параметрами.
 - b) Методи:
 - Перевизначення getType – повертає вид страви.
- 8) Клас «Beverage», нащадок класу «Dish» – напій:
- a) Конструктори:
 - Конструктор з параметрами.
 - b) Методи:
 - Перевизначення getType – повертає вид страви.
- 9) Клас «Memory» – клас для безпечного зберігання об'єктів:
- a) Статичні приватні поля: visitors, sellers, dishes, selectedVisitor, selectedSeller.
 - b) Методи:
 - addVisitor – додає відвідувача;
 - addSeller – додає продавця;
 - addDish – додає страву;

- deleteVisitor – видаляє відвідувача;
- deleteSeller – видаляє продавця;
- deleteDish – видаляє страву;
- getVisitor – повертає відвідувача за індексом;
- getSeller – повертає продавця за індексом;
- getDish – повертає страву за індексом;
- getVisitorCount – повертає кількість відвідувачів;
- getSellerCount – повертає кількість покупців;
- getDishCount – повертає кількість об'єктів страв;
- selectVisitor – змінює вибраного відвідувача за індексом;
- selectSeller – змінює вибраного продавця за індексом;
- getSelectedVisitor – повертає вибраного відвідувача;
- getSelectedSeller – повертає вибраного продавця;
- nullSeller – прибирає вибір продавця;
- returnDishes – повертає всі страви;
- returnSellers – повертає всіх продавців.

2.2 Інтерфейс

Готовий проєкт розміщено в [4]. Під час запуску програми користувач бачить початковий інтерфейс (Рис 2.1).

```

-----Інтерфейс-----
1. Вибрати покупця
2. Вибрати продавця
3. Показати меню
4. Закінчити день
7. Додати блюдо
8. Додати людину
9. Видалити людину
0. Вихід

Ваш вибір: |

```

Рисунок 2.1 – Початковий інтерфейс

Користувач потрапляє сюди коли запускає програму, виходить з вибору дії або завершає цю дію. Після того як користувач вибере дію яку хоче зробити, йому буде виведено інформацію стосовно неї. Якщо користувач зразу вибере 3 варіант, попередньо не вибрав покупця чи продавця, буде виведено помилку і початковий інтерфейс (Рис 2.2, 2.3).

```

Ваш вибір: 3

[Спочатку виберіть покупця!]
-----Інтерфейс-----
1. Вибрати покупця
2. Вибрати продавця
3. Показати меню
4. Закінчити день
7. Додати блюдо
8. Додати людину
9. Видалити людину
0. Вихід

```

Рисунок 2.2 – Помилка коли не вибраний покупець

```

Ваш вибір: 3

[Спочатку виберіть продавця!]
-----Інтерфейс-----
1. Вибрати покупця (Вибраний:
2. Вибрати продавця
3. Показати меню
4. Закінчити день
7. Додати блюдо
8. Додати людину
9. Видалити людину
0. Вихід

```

Рисунок 2.3 – Помилка коли не вибраний продавець

Якщо користувач вибере 1 варіант, буде виведено усіх покупців для вибору (Рис 2.4) і опція повернутися до початкового інтерфейсу.

```

Ваш вибір: 1

1. Коваленко Максим Андрійович, грошей: 550 грн
2. Шевченко Софія Олегівна, грошей: 400 грн
3. Богатій Дмитро Вікторович, грошей: 90000 грн
0. Вихід

```

Рисунок 2.4 – Вибір покупця

При виборі одного з покупців користувача поверне до початкового інтерфейсу, але тепер біля опції вибору покупців буде відображено нинішній вибір (Рис 2.5).

```

-----Інтерфейс-----
1. Вибрати покупця (Вибраний: Коваленко Максим Андрійович, грошей: 550 грн)
2. Вибрати продавця
3. Показати меню
4. Закінчити день
7. Додати блюдо
8. Додати людину
9. Видалити людину
0. Вихід

```

Рисунок 2.5 – Початковий інтерфейс з вибраним покупцем

Вибір продавця проходить так само (Рис 2.6, 2.7).

```
Ваш вибір: 2

1. Ковальчук Софія Романівна, шанс порушення: 5%
2. Шахрай Андрій Вікторович, шанс порушення: 90%
0. Вихід
```

Рисунок 2.6 – Вибір продавця

```
-----Інтерфейс-----
1. Вибрати покупця (Вибраний: Коваленко Максим Андрійович, грошей: 550 грн)
2. Вибрати продавця (Вибраний: Ковальчук Софія Романівна, шанс порушення: 5%)
3. Показати меню
4. Закінчити день
7. Додати блюдо
8. Додати людину
9. Видалити людину
0. Вихід
```

Рисунок 2.7 – Початковий інтерфейс з вибраним продавцем

Тепер у випадку вибору 3 користувача зустрине меню (Рис 2.8) з опціями вибору страви та опцією виходу до початкового інтерфейсу.

```
1. [Гаряча страва] Борщ, ціна: 150 грн, кількість: 25 порцій
2. [Холодна страва] Цезар, ціна: 270 грн, кількість: 20 порцій
3. [Напій] Кава, ціна: 30 грн, кількість: 35 порцій
0. Вихід
У покупця грошей: 550 грн, замовлення: 0 грн
```

Рисунок 2.8 – Меню

При виборі страви також зазначається кількість порцій (Рис 2.9).

```
Що купуєте: 1
Скільки порцій ви хочете купити? |
```

Рисунок 2.9 – Вибір кількості порцій

Якщо вибрати більше порцій чим є, буде виведена помилка (Рис 2.10).

Скільки порцій ви хочете купити? 30

[Недостатньо порцій!]

Рисунок 2.10 – Помилка коли недостатньо порцій

Після коректного вибору страви та кількості порцій користувач зможе продовжувати замовлення чи завершити його (Рис 2.11). Також кількість страв буде тимчасово зменшена. А вартість замовлення збільшиться.

Скільки порцій ви хочете купити? 15

1. [Гаряча страва] Борщ, ціна: 150 грн, кількість: 10 порцій
 2. [Холодна страва] Цезар, ціна: 270 грн, кількість: 20 порцій
 3. [Напій] Кава, ціна: 30 грн, кількість: 35 порцій
 4. Зробити замовлення
 0. Вихід
- У покупця грошей: 550 грн, замовлення: 2250 грн

Рисунок 2.11 – Меню з опцією завершення замовлення

Якщо зробити замовлення коли у відвідувача недостатньо коштів, буде виведено помилку (Рис 2.12) з можливістю скоригувати замовлення чи повністю закрити меню з поверненням до початкового інтерфейсу.

[Недостатньо коштів!]

1. Зробити замовлення ще раз
2. Вихід

Рисунок 2.12 – Помилка коли недостатньо коштів

Якщо коштів вистачило, то замовлення завершиться (Рис 2.13): кількість порцій і кількість коштів у користувача зменшиться, кількість коштів у касі збільшиться з шансом (Рис 2.14) того, що продавець привластнить собі частину коштів.

[Куплено]
Борщ: 2 порцій

Рисунок 2.13 – Завершення замовлення

Вибраний: Шахрай Андрій Вікторович , шанс порушення: 90%

Рисунок 2.14 – Шанс недоброчесності продавця

Якщо в початковому інтерфейсі вибрати варіант 4, то день завершиться і прийде інспектор (Рис 2.15).

Ваш вибір: 4

[День завершився]
Інспектор Козак Максим Сергійович (Благородне товариство) перевіряє...

Рисунок 2.15 – Завершення дня

Інспектор перевіряє кількість проданих страв та кількість грошей в касі. Якщо порушень не знайдено (Рис 2.16), то день завершується звичайним чином і кількість страв поповнюється (Рис 2.17).

Інспектор Козак Максим Сергійович (Благородне товариство) перевіряє...
 Всього продано: 3
 Всього зроблено: 50
 Всього залишилось: 47
 Інспектор не виявив зникнення товару
 Інспектор перевіряє касу...
 Повинно бути: 690 грн
 В касі: 690 грн
 Всі продавці були чесними. Інспектор задоволений.

Рисунок 2.16 – Перевірка інспектора без вияву порушень

[Новий день]
 Кількість їжі поповнило

Рисунок 2.17 – Новий день

Якщо ж інспектор виявляє порушення, починається перевірка кожного продавця з послідуєчим арестом (Рис 2.18) і неможливість продажі страв заарештованим (Рис 2.19).

Повинно бути: 600 грн
 В касі: 585 грн
 Інспектор не задоволений. В касі не хватає грошей
 Інспектор шукає злодія...
 Злочинець 1: Шахрай Андрій Вікторович – вкрав: 15 грн --- [АРЕШТОВАН]
 [Всі злодії спіймані]

Рисунок 2.18 – Перевірка інспектора з виявом порушень

1. Ковальчук Софія Романівна, шанс порушення: 5%
 2. Шахрай Андрій Вікторович, шанс порушення: 90% [Арештован]
 0. Вихід
 Ваш вибір: 2
 [У злодіїв нічого купувати не можна!]

Рисунок 2.19 – Спроба вибрати заарештованого продавця

Також в користувача є опції створення своїх страв (Рис 2.20) та додання людей (Рис 2.21) чи видалення (Рис 2.22).

```
Ваш вибір: 7

1. Гаряча страва
2. Холодна страва
3. Напій
0. Вихід

Ваш вибір: 1

Введіть назву: Юшка

Введіть вартість: 150

Введіть кількість: 15

Створено: [Гаряча страва] Юшка, ціна: 150 грн, кількість: 15 порцій
```

Рисунок 2.20 – Створення страви

```
Ваш вибір: 8

1. Додати покупця
2. Додати продавця
0. Вихід

Ваш вибір: 1

Введіть ПІБ: Іванов Іван Іванович

Введіть кількість грошей: 100

Додано: Іванов Іван Іванович, грошей: 100 грн
```

Рисунок 2.21 – Додання людини

```

Ваш вибір: 9

1. Видалити покупця
2. Видалити продавця
0. Вихід

Ваш вибір: 1

1. Коваленко Максим Андрійович, грошей: 550 грн
2. Шевченко Софія Олегівна, грошей: 400 грн
3. Богатій Дмитро Вікторович, грошей: 90000 грн
4. Іванов Іван Іванович, грошей: 100 грн
0. Вихід

Ваш вибір: 4

Видалено успішно!

```

Рисунок 2.22 – Видалення людини

Також під час будь-якого вводу відбувається дві перевірки: чи є введений символ числом (Рис 2.23) та чи можливий це варіант (Рис 2.24).

```

Ваш вибір: нечисло

[Введіть число!]

```

Рисунок 2.23 – Перевірка на число

```

Ваш вибір: 5

[Виберіть коректний варіант!]

```

Рисунок 2.24 – Перевірка на можливість варіанту дії

ВИСНОВКИ

Результатом курсової роботи стало створення консольної програми «Буфет» мовою C++ із застосуванням об'єктно-орієнтованого програмування. Архітектура проєкту містить ієрархічну систему класів, що описують страви, меню, відвідувачів і продавців, а також забезпечують їхню коректну взаємодію.

Під час проектування було практично застосовано інкапсуляцію, поліморфізм та наслідування. Кожен компонент програми містить набір конструкторів, полів і методів. Серед головних функцій реалізовано: консольне керування, створення об'єктів людей і страв, перевірку коректності введення інформації, а також функцію інспекторського контролю за порушеннями.

Цей проєкт дозволив покращити практичні вміння роботи з мовою C++, базовими конструкціями класів, методами та консоллю. Написана програма підтверджує, що засоби ООП є чудовим інструментом для побудови повноцінних систем керування об'єктами громадського харчування.

СПИСОК ЛІТЕРАТУРИ

1. Об'єктно-орієнтоване програмування: [Підручник] / В.В. Бублик [Електронне видання] / Київ, 2026. – Режим доступу: https://itknyga.com.ua/documents/OOP_final.pdf
2. Web Library. Основні поняття ООП. [Електронне видання] – Режим доступу: <http://weblib.com.ua/blog/osnovni-ponyatty-a-oor>
3. Репозиторій GitHub на курсовий – Режим доступу: <https://github.com/Yanchik228/CourseProject>

Додаток А. Діаграма класів

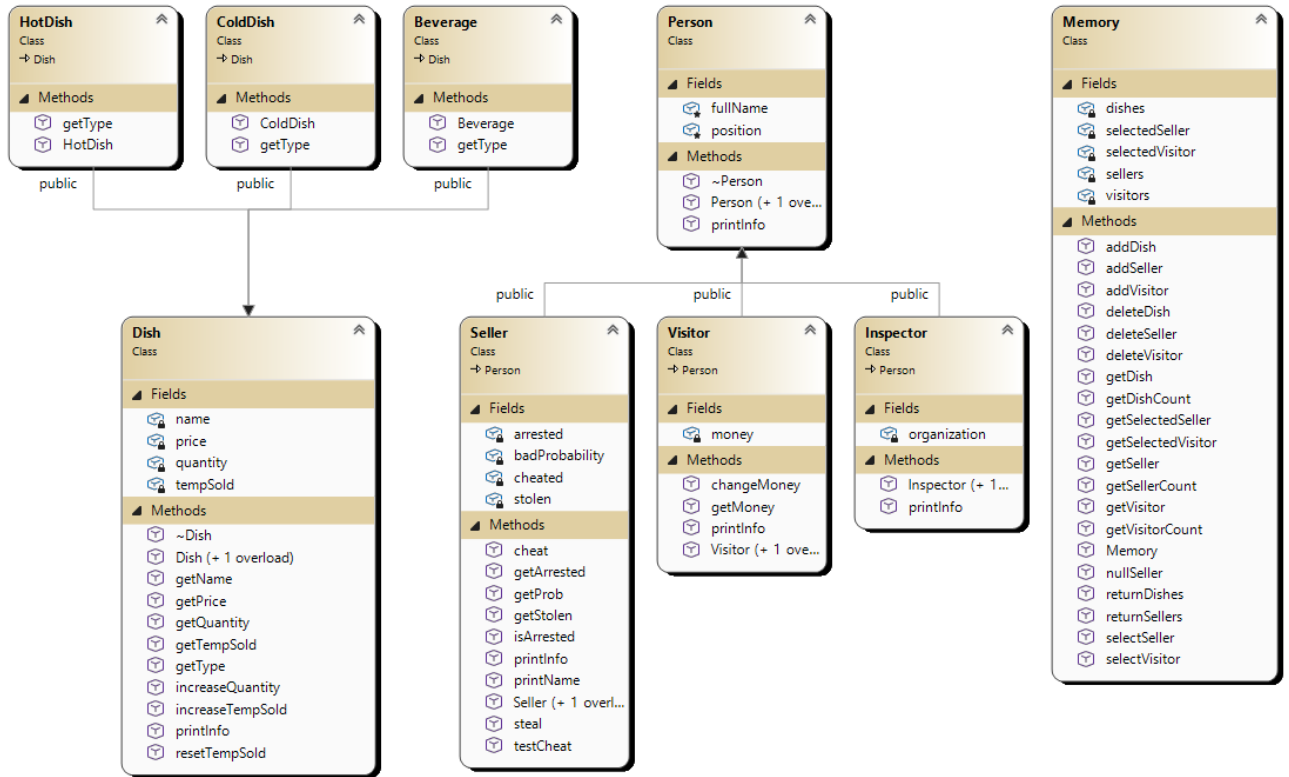


Рисунок А.1 – Діаграма класів

Додаток Б. Код основних класів

```
#include "Person.h"
#include <stdexcept>
#include <iostream>

Person::Person()
    : fullName("Невідомо") {}
Person::Person(const std::string& name)
    : fullName(name)
{
    if (fullName.empty()) {
        throw std::invalid_argument("ПІБ не може бути порожнім!");
    }
}
```

Лістинг Б.1 – Клас Person

```
#include "Visitor.h"
#include <iostream>

Visitor::Visitor()
    : Person("Невідомо"), money(0) {}
Visitor::Visitor(const std::string& name, int money)
    : Person(name), money(money)
{
    if (money < 0) {
        throw std::invalid_argument("Грошей не може бути від'ємна кількість!");
    }
}

int Visitor::getMoney() const {
    return money;
}

void Visitor::changeMoney(int n) {
    money += n;
}

void Visitor::printInfo() const {
    std::cout << fullName << ", " << "грошей: " << money << " грн";
}
```

Лістинг Б.2 – Клас Visitor

```

#include "Seller.h"
#include <stdexcept>
#include <iostream>

Seller::Seller()
    : Person("Невідомо"), badProbability(0.0), arrested(false), cheated(false),
  stolen(0) {}
Seller::Seller(const std::string& name, double prob)
    : Person(name), badProbability(prob), arrested(false), cheated(false), stolen(0)
{
    if (badProbability > 1.0 || badProbability < 0.0) {
        throw std::invalid_argument("Неможливий шанс!");
    }
}

double Seller::getProb() const {
    return badProbability;
}

int Seller::getStolen() const {
    return stolen;
}

bool Seller::isArrested() const {
    return arrested;
}

bool Seller::testCheat() const {
    return cheated;
}

void Seller::cheat() {
    cheated = true;
}

void Seller::steal(int n) {
    stolen += n;
}

void Seller::getArrested() {
    arrested = true;
}

void Seller::printName() const {
    std::cout << fullName;
}

void Seller::printInfo() const {
    std::cout << fullName << ", " << "шанс порушення: " << badProbability * 100 << "%";
    if (arrested) {
        std::cout << " [Арештован]";
    }
}

```

Лістинг Б.3 – Клас Seller

```

#include "Inspector.h"
#include <stdexcept>
#include <iostream>

Inspector::Inspector()
    : Person("Невідомо"), organization("Немає") {}
Inspector::Inspector(const std::string& name, const std::string& org)
    : Person(name), organization(org)
{
    if (organization.empty()) {
        throw std::invalid_argument("Поле організації не може бути порожнім!");
    }
}

void Inspector::printInfo() const {
    std::cout << fullName << " (" << organization << ")";
}

```

Лістинг Б.4 – Клас Inspector

```

#include "Dish.h"
#include <stdexcept>
#include <iostream>
#include <cmath>

Dish::Dish()
    : name("Невідомо"), price(0), quantity(0), tempSold(0) {}
Dish::Dish(const std::string& name, int price, int quantity)
    : name(name), price(price), quantity(quantity), tempSold(0)
{
    if (name.empty()) {
        throw std::invalid_argument("Назва не може бути порожнім!");
    }
    if (price < 0) {
        throw std::invalid_argument("Вартість не може бути негативною!");
    }
    if (quantity < 0) {
        throw std::invalid_argument("Кількість не може бути негативною!");
    }
}

std::string Dish::getName() const {
    return name;
}
int Dish::getPrice() const {
    return price;
}
int Dish::getQuantity() const {
    return quantity;
}
int Dish::getTempSold() const {
    return tempSold;
}

```

Лістинг Б.5 – Клас Dish, аркуш 1


```

void Dish::increaseQuantity(int increase) {
    if (increase <= 0) return;
    else {
        quantity += increase;
    }
}
void Dish::increaseTempSold(int increase) {
    if (increase <= 0) return;
    else {
        tempSold += increase;
        quantity -= increase;
    }
}
void Dish::resetTempSold() {
    tempSold = 0;
}

void Dish::printInfo() const {
    std::cout << "[" << getType() << "]" " << name << ", ціна: " << price << " грн,
    кількість: " << quantity << " порцій";
}

```

Лістинг Б.5 – Клас Dish, аркуш 2

```

HotDish::HotDish(const std::string& name, int price, int quantity)
    : Dish(name, price, quantity) {}
std::string HotDish::getType() const {
    return "Гаряча страва";
}

```

Лістинг Б.6 – Клас HotDish

```

ColdDish::ColdDish(const std::string& name, int price, int quantity)
    : Dish(name, price, quantity) {}
std::string ColdDish::getType() const {
    return "Холодна страва";
}

```

Лістинг Б.7 – Клас ColdDish

```

Beverage::Beverage(const std::string& name, int price, int quantity)
    : Dish(name, price, quantity) {}
std::string Beverage::getType() const {
    return "Напій";
}

```

Лістинг Б.8 – Клас Beverage

```

#include "Memory.h"

std::vector<std::shared_ptr<Visitor>> Memory::visitors;
std::vector<std::shared_ptr<Seller>> Memory::sellers;
std::vector<std::shared_ptr<Dish>> Memory::dishes;

std::shared_ptr<Visitor> Memory::selectedVisitor = nullptr;
std::shared_ptr<Seller> Memory::selectedSeller = nullptr;

void Memory::addVisitor(const std::shared_ptr<Visitor>& visitor) {
    if (visitor != nullptr) {
        visitors.push_back(visitor);
    }
}

void Memory::addSeller(const std::shared_ptr<Seller>& seller) {
    if (seller != nullptr) {
        sellers.push_back(seller);
    }
}

void Memory::addDish(const std::shared_ptr<Dish>& dish) {
    if (dish != nullptr) {
        dishes.push_back(dish);
    }
}

bool Memory::deleteVisitor(size_t index) {
    if (index < visitors.size()) {
        visitors.erase(visitors.begin() + index);
        return true;
    }
    return false;
}

bool Memory::deleteSeller(size_t index) {
    if (index < sellers.size()) {
        sellers.erase(sellers.begin() + index);
        return true;
    }
    return false;
}

bool Memory::deleteDish(size_t index) {
    if (index < dishes.size()) {
        dishes.erase(dishes.begin() + index);
        return true;
    }
    return false;
}

std::shared_ptr<Visitor> Memory::getVisitor(size_t index) {
    if (index < visitors.size()) {
        return visitors[index];
    }
    return nullptr;
}

std::shared_ptr<Seller> Memory::getSeller(size_t index) {
    if (index < sellers.size()) {
        return sellers[index];
    }
    return nullptr;
}

```

```

std::shared_ptr<Dish> Memory::getDish(size_t index) {
    if (index < dishes.size()) {
        return dishes[index];
    }
    return nullptr;
}

size_t Memory::getVisitorCount() {
    return visitors.size();
}
size_t Memory::getSellerCount() {
    return sellers.size();
}
size_t Memory::getDishCount() {
    return dishes.size();
}

void Memory::selectVisitor(size_t index) {
    if (index < visitors.size()) {
        selectedVisitor = visitors[index];
    }
}
void Memory::selectSeller(size_t index) {
    if (index < sellers.size()) {
        selectedSeller = sellers[index];
    }
}
void Memory::nullSeller() {
    selectedSeller = nullptr;
}

std::shared_ptr<Visitor> Memory::getSelectedVisitor() {
    return selectedVisitor;
}
std::shared_ptr<Seller> Memory::getSelectedSeller() {
    return selectedSeller;
}

std::vector<std::shared_ptr<Dish>> Memory::returnDishes() {
    return dishes;
}
std::vector<std::shared_ptr<Seller>> Memory::returnSellers() {
    return sellers;
}

```

Лістинг Б.9 – Клас Memory, аркуш 2